Hub-less Cross-Honeypot Threat Intelligence: Federated IOC Corroboration on a Shared Ledger

Suhas Reddy Annem[1], Radhika G[2]
Department of Computer Science and Engineering
Amrita School of Computing, Coimbatore
Amrita Vishwa Vidyapeetham, India
cb.sc.u4cse24503@cb.students.amrita.edu[1], g_radhika@cb.amrita.edu[2]

***Abstract*—A scanning campaign rarely stops at one target. It sweeps address ranges and touches many decoys in turn, so any single honeypot records only a fragment of what happened. The usual remedy is a central threat-intelligence server that every sensor reports into. That remedy carries its own cost: the server becomes the one machine an adversary must reach to blind the whole network, and the one machine that must stay up for correlation to occur. We propose and measure an alternative in which no such server exists. Each node hashes the indicators of compromise (IOCs) it observes, tags the digest with its own identity, and writes it to a blockchain shared by all nodes. No node calls another node's API, and no process coordinates the group. Corroboration, the recognition that two nodes saw the same campaign, is then obtained by reading the chain rather than by trusting a hub's database. We evaluate the design on a live Ganache chain using five concurrent honeypot node workers, each observing 60 IOCs, for 300 anchored events, with a 35% ground-truth overlap drawn from a 25-IOC campaign pool. Anchoring costs a mean of 8.44 ms (median 8.11 ms, p95 11.06 ms), and per-node means stay within a 7.93-9.19 ms band, showing no write contention at this scale. A node polling the chain with no coupling to the writer observes a new anchor after a mean of 1.94 ms (p95 2.92 ms), succeeding in all 40 trials. A fresh node that has never contacted the other five recovers all 220 distinct IOCs from chain history alone, with correct provenance, and separates the 195 single-node indicators from the 25 seen by two or more nodes with no false merges and no missed corroborations; since shared indicators are byte-identical by construction, this verifies the reconstruction pipeline rather than a fuzzy-matching capability. We then rewrite one node's local log, as an attacker who had breached that honeypot would, and the forgery is caught: the original digest still stands on the ledger, and the forged digest appears nowhere. Finally, five nodes writing at once sustain 228.58 anchored events per second, against 98.57 for a single sequential writer measured in the same harness through the same code path, a 2.32-fold gain from concurrency alone. We report these figures alongside what a single local chain cannot establish about a real multi-organization deployment.**

***Index Terms*—federated threat intelligence, honeypot, blockchain, decentralized coordination, IOC sharing, Byzantine fault tolerance, cyber-deception, MISP, STIX/TAXII**

I. Introduction

Intrusion campaigns are seldom bespoke. A credential-stuffing botnet, a worm scanning for an exposed service, or a crew reusing one exploit kit will produce closely related indicators against every honeypot it touches: the same source addresses, the same payload strings, the same command sequences. Any one decoy, however well instrumented, sees only its own slice. The standard response to that fragmentation has been to centralize. An organization stands up a threat-intelligence platform, has every sensor report into it, and lets the platform correlate. MISP [1] and the STIX/TAXII standards [2], [3] formalize this pattern, supplying a common data model and a transport for moving indicators between a sensor and a sharing hub.

Centralizing correlation solves fragmentation but creates a dependency. The aggregator becomes a single point of trust, since every operator must believe it will not lose, alter, or quietly withhold what it is given. It is also a single point of failure and an obvious target. An adversary who locates the sharing hub gains one asset whose compromise silences correlation across the whole sensor network, and a hub taken down for maintenance leaves every participant temporarily blind. The dependency sits awkwardly in a honeypot network, where the decoys exist to be attacked but the machine correlating the evidence is not hardened against the same adversary.

This paper asks whether the dependency is necessary. Can honeypot nodes deployed independently, possibly by different organizations, build a correct, deduplicated, cross-corroborated view of a campaign without any node trusting, calling, or even knowing about another? Our design uses a blockchain that all nodes write to as an ownerless bulletin board. Each node observes its own IOCs privately, hashes them, and anchors each digest tagged only with its own identifier. Raw indicator content never leaves the node, and no request is ever sent to a peer.

Anchoring a digest on a ledger is a familiar way to protect one machine's own log: a collector commits a hash of what it captured, so a later rewrite becomes detectable. That guards a single node's evidence against that node's compromise, and it is not what we study here. Our question is what several honeypots, each anchoring independently, can jointly establish about a campaign none of them saw in full. Our contributions are:

1) A hub-less design for cross-honeypot IOC sharing in which the chain serves only as a shared append point. It is not a coordination service, not a smart-contract registry, and never a place where one node reads another node's API, so a fresh node bootstraps by reading the chain and doing nothing else.
2) A node-identity tagging and reconstruction scheme that

lets any reader recover both provenance, meaning which node saw an indicator, and corroboration, meaning how many independent nodes saw it, from transaction history alone.

3) A measured evaluation on a live Ganache deployment with five concurrent node workers, covering per-node and aggregate anchor latency, propagation latency measured with a blind observer, corroboration precision and recall against a known overlap, a compromised-node forgery experiment, and concurrent throughput against a sequential baseline measured in the same harness.
4) An account of what a single local chain with simulated nodes does not demonstrate about a geographically distributed, multi-organization deployment on a network with real consensus latency and real partitions.

Section 2 situates the work against centralized and federated platforms, blockchain-based sharing designs, and blockchain-honeypot coordination schemes. Section 3 describes the node model, the tagging scheme, and the reconstruction and detection algorithms. Section 4 covers methodology, Section 5 the results, Section 6 the limitations, and Section 7 concludes.

II. Related Work

A. Centralized and federated threat-intelligence platforms

MISP [1] is the de facto open-source standard for sharing structured indicators, and it supports a federated topology in which multiple instances synchronize. That synchronization still assumes each instance is a trusted, administered node with which peers explicitly configure a sync relationship. Trust is established out of band between operators, and a new participant must be admitted to that web of relationships before it receives anything. STIX [2] and TAXII [3] standardize the data model for an indicator and the client-server transport for publishing and polling a feed. A TAXII deployment is a set of servers, or collections, that clients push to and pull from; a new client must know which server to address and be authorized there. Both ecosystems solve interoperability, but neither removes the requirement that some administered node is the rendezvous point through which sharing occurs.

The difficulty is organizational as much as technical. A mixed-methods study of MISP users by Stojkovski and colleagues [4] found that establishing and reasoning about trust relationships is a persistent source of friction, which is the coordination burden a hub-less design removes. Our design differs in kind rather than degree. There is no server a new node must be introduced to, no sync relationship to configure, and no operator positioned to omit another node's contribution from what a third party sees. Every node writes to the same ledger, and every reader reads it. The trust assumption moves off any single operator and onto the ledger's consensus.

B. Blockchain-based threat-intelligence sharing

Distributed ledgers have already been proposed as a substrate for CTI exchange. A recent review by Ruggiero and Bonomi [5] catalogues the requirements and residual threats of this design space. TRADE [6] builds a peer-to-peer, privacy-preserving exchange in which the producer of an indicator retains control over its dissemination. TrustShare [7] targets secure and trusted sharing, pairing on-chain records with access-control machinery so participants exchange intelligence without a single administrative authority. Dunnett and colleagues survey the challenges of applying blockchain to CTI sharing [8] and propose a verifiable, differential framework in which contributions can be validated and selectively disclosed [9]. Others pair a ledger with federated learning so participants share detection models instead of raw indicators [10], and a survey by Tariq and colleagues [11] places these approaches in the wider distributed-ledger landscape.

Our design is more minimal than most of this work. The systems above generally put logic on-chain: contracts that score reputation, enforce access policy, judge contribution quality, or coordinate a training round. Such logic is capable, but it widens the trusted surface, because participants must also agree the contract encodes the right policy, and it usually assumes an enrollment step admitting a node to a consortium. We ask a narrower question. How much of a hub's function survives if the chain carries nothing but tagged hashes, with no contract logic, no reputation state, no access control, and no enrollment? Our measurements indicate that deduplication, corroboration, and compromised-node detection all survive, which suggests these properties do not require the machinery surrounding work assumes.

C. Byzantine fault tolerance and decentralized coordination

How mutually distrusting parties agree on a shared history without a coordinator is the classical Byzantine fault tolerance problem [16]. The base chain's consensus layer solves it so our nodes need not. We neither implement nor evaluate a BFT protocol. Ganache's development chain supplies ordering and finality by fiat rather than by running consensus among validators. The design we evaluate is nonetheless the kind of application that sits atop a BFT-secured public or consortium chain: many mutually distrusting writers, none trusting another, and readers who trust the chain's consensus rather than any participant. Section 6 states what this substitution defers.

D. Honeypots as distributed intelligence sensors

Honeypots have long served as sensors whose traffic is entirely suspect [13], and surveys of honeypot software describe the deployment models and analysis pipelines built around them [14]. Two findings motivate correlating across many sensors. An individual low- or medium-interaction decoy observes only the fraction of a campaign that reaches it. And, as Vetterl and Clayton showed at Internet scale, such decoys are systematically fingerprintable, so a careful attacker can recognize and avoid one [15]. Closer to our setting, Otoum and colleagues combine a blockchain with adaptive honeypots for IoT security, using the ledger as a trust-aware coordination layer among devices [12].

That work treats the chain as an active coordination and trust-arbitration mechanism, with nodes reading and writing

TABLE I

How this work is positioned against centralized and federated threat-intelligence platforms and against prior blockchain-based sharing and blockchain-honeypot coordination designs.

System / class	No trusted coordinator	New node bootstraps from chain alone	Measured cross-node corroboration	Compromised-node detection via peers
MISP federated instances [1], [4] (configured sync relationships)	○	○	○	○
STIX/TAXII hub-and-collection [2], [3] (client-server feeds)	○	○	○	○
Blockchain CTI sharing with on-chain policy / reputation [6]-[10] (contract-mediated trust logic)	●	○	○	○
Blockchain-coordinated honeypots [12] (trust arbitration between sensors)	●	○	○	○
This work (hash-only passive bulletin board)	●	●	●	●

contract state encoding how far to trust a peer's report. Our nodes do not coordinate. They exchange no messages, consult no reputation score, and are unaware of one another. Deduplication, corroboration, and compromised-node detection are all recovered by a reader scanning raw transaction history, which keeps the trust surface at the chain's own consensus.

III. System Design

A. Node model

We model a federated deployment as N independently operated honeypot nodes. Each runs its own instance of the five-service decoy stack described above and observes its own stream of attacker interactions, from which it derives IOC records. In our implementation an IOC record comprises the attacker source address and the observed payload. Nodes are not assumed to trust each other, are not assumed to be mutually reachable, and never issue a request to one another. Read and write access to the same ledger is all they share.

B. IOC hashing and provenance tagging

For each IOC, a node canonically serializes the record, using deterministic key ordering so the byte representation does not depend on field insertion order, and computes its SHA-256 digest [18]. It then concatenates its own node identifier with that digest as node_id:hash and writes the result into the input-data field of a transaction sent to the shared chain. No raw indicator content reaches the chain: no source address, no payload string, only the digest and the writer's self-declared identity. Anchored records therefore stay small and cheap, and the chain never becomes a public repository of the sensitive traffic a decoy captured. It records only that some node observed something hashing to a particular value.

C. The chain as a passive bulletin board

The chain has no role beyond ordering transactions and retaining them immutably. There is no smart contract, no on-chain registry of known nodes, and no logic executed at write time beyond a standard transfer carrying tagged input data. The minimalism is the point. Every property the system provides can be recovered by applying a fixed decoding rule to raw chain history, so an independent client, a new implementation, or an auditor without special software reaches the same conclusions as a purpose-built tool.

D. Reconstruction: how a fresh node rebuilds the global picture

A node that has never interacted with the network bootstraps with one procedure. It scans every block, and for each transaction whose input data decodes to a well-formed node_id:hash tag, it records that node_id as an observer of that hash. The result is a map from IOC digest to the set of node identifiers that anchored it. An IOC counts as corroborated, meaning it belongs to a multi-target campaign rather than an isolated local event, when its observer set has size two or more.

That is the whole consensus mechanism. There is no voting and no threshold beyond set cardinality, and no trust is placed in any node's self-report. The corroboration signal is one hash appearing under two independently generated transactions from two node identities, which a single compromised node cannot manufacture alone because it cannot forge another node's signing key.

E. Compromised-node detection via the shared ledger

Each node also keeps a local record of the IOCs it anchored, for its own operational use. A discrepancy between that local record and what the chain says the node anchored is itself a signal.

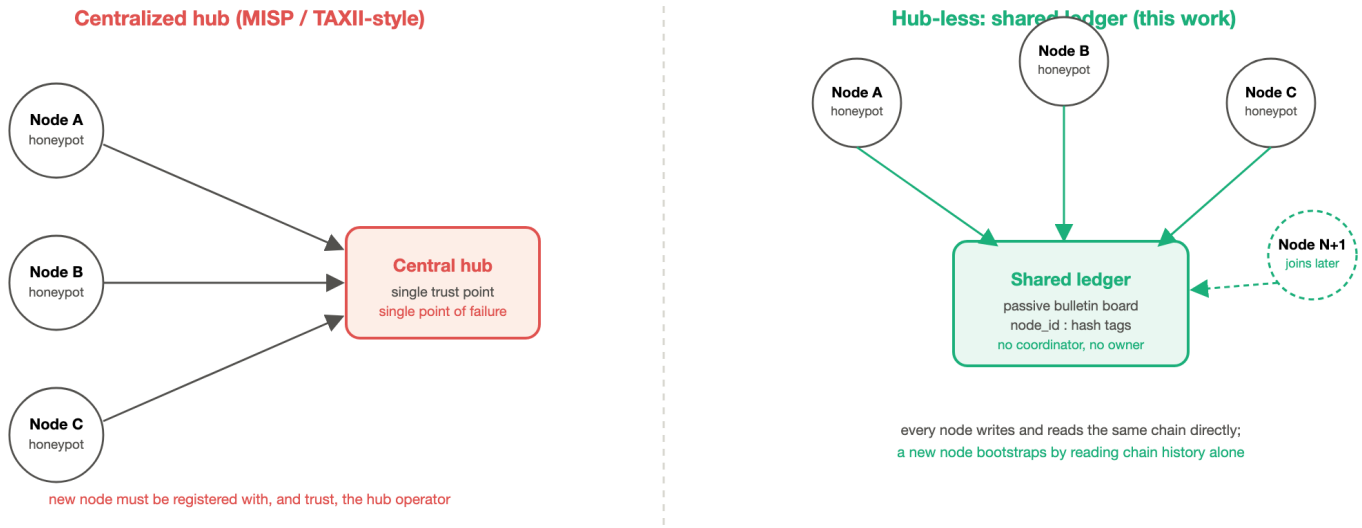


Fig. 1. Centralized threat-intel sharing (MISP/TAXII-style) versus the hub-less design evaluated in this paper. Every node writes directly to a shared ledger with no coordinating hub; a new node bootstraps by reading chain history alone.

Suppose an attacker breaches node k and rewrites its local log, altering a captured payload string to disguise an interaction. The rewritten record now hashes to a different digest than the one anchored at the time of observation. The original digest remains on the chain, tagged with node k's identity, because the anchor transaction was mined before the breach and no node, compromised or not, can edit a transaction already included. The forged digest was never anchored by anyone, so it is absent. Any party, whether another node, a third-party auditor, or node k's own operator during recovery, detects the tampering from the mismatch.

Detection here does not require another node to have seen the same IOC. It requires only the victim's own prior anchor, which an attacker arriving after that anchor was mined cannot undo.

IV. Methodology

All results come from a benchmark harness (benchmark_federation.py) driving a live Ganache [19] development chain, an Ethereum-compatible ledger implementing the semantics of the Ethereum yellow paper [17]. Five concurrent workers stand in for five separately deployed honeypot nodes, each writing through its own dedicated chain account. No reported number is estimated or hand-entered, and every figure in this paper comes from this one harness; no value is imported from any other benchmark. The harness performs seven measurements.

(1) Per-node and aggregate anchor latency. Five IOC streams are constructed with a controlled overlap. A pool of 25 campaign IOCs is generated, and each node's 60-IOC stream draws 35% of its entries from that pool, with the remainder unique to the node, shuffled so anchoring order gives no hint of which entries are shared. Each node anchors its stream sequentially. The wall-clock time from submitting a transaction to observing its mined receipt is recorded per event, per node, and in aggregate over all 300 events.

(2) Cross-node propagation latency. To measure how quickly an anchor becomes visible to a party that did not write it and has no channel to the writer, the harness submits a tagged transaction and records the moment it is mined. It then polls the chain by block number, with no shared memory or signal from the writing step, until it observes the expected hash in a transaction's input data. The interval between mining and observation is the propagation latency, repeated over 40 trials.

(3) Reconstruction and corroboration. Once anchoring completes, the harness runs the fresh-node bootstrap of Section 3.4. It scans every block produced during the anchoring phase, decodes each node_id:hash tag, and builds the observer-set map from scratch, retaining nothing about which node wrote which transaction beyond what the chain yields. The result is compared against the union of all local IOCs across the five nodes, giving recall of the full dataset, and against the known set of 25 shared IOCs, giving precision and recall of the corroboration signal specifically.

(4) Compromised-node resilience. One node's local copy of an already-anchored IOC is mutated after the fact by altering its payload string, simulating an attacker who breached that honeypot and rewrote its evidence. This produces a different hash from the one anchored. The harness checks that the original hash is still on-chain under node0's identity, which it must be since the anchor predates the simulated compromise, and that the forged hash appears nowhere, which it must not since no one anchored it. Detection counts as successful only when both conditions hold.

(5) Sequential single-writer baseline. One writer anchors 100 events sequentially through the same anchor-and-confirm code path, on the same chain, inside the same process. This is the reference point for the concurrent measurement, so the

comparison varies only the number of writers.

(6) Concurrent-node throughput. Five node workers anchor 20 events each against the same chain concurrently, and the harness measures aggregate wall-clock throughput as confirmed events divided by elapsed time. Each worker uses a distinct chain account, which avoids the nonce contention that would otherwise serialize writers sharing one account.

(7) Centralized-aggregator contrast. For contrast we name, without benchmarking through a live failure injection, the single-point-of-failure property of a naive centralized model in which all nodes POST their IOCs to one in-memory aggregator. Killing that process loses whatever has not been flushed to durable storage, and no other party can reconstruct the picture in its absence. We report this as a structural property rather than a timing result, since a fair timing comparison would require fixing an aggregator implementation and a failure-injection protocol beyond this paper's scope.

IOC payloads are synthetic. They are drawn from a fixed pool of hostile-looking strings, including shell commands, SQL-injection attempts, wallet-draining JSON-RPC calls, and reconnaissance commands, paired with randomized source addresses. They are not captured attacker traffic, and the decoy services themselves are not exercised: the harness writes to the chain directly. What is measured here is the anchoring and reconstruction machinery, not the decoys that would feed it in deployment.

V. Results

A. Anchor latency across five independent nodes

Over all 300 anchored events, 60 per node across 5 nodes, the aggregate mean anchor latency is **8.44 ms**, with a median of 8.11 ms, a p95 of 11.06 ms, and a standard deviation of 2.72 ms. The fastest anchor completed in 6.03 ms and the slowest in 45.13 ms. Per node, the means fall in a 7.93-9.19 ms band: node0 at 9.19 ms, node1 at 8.22 ms, node2 at 8.66 ms, node3 at 8.19 ms, and node4 at 7.93 ms. Node medians range from 7.73 ms to 8.50 ms. The spread is tight enough to indicate no meaningful write contention at this scale; from any one node's perspective the shared chain behaves much as it would for a writer working alone.

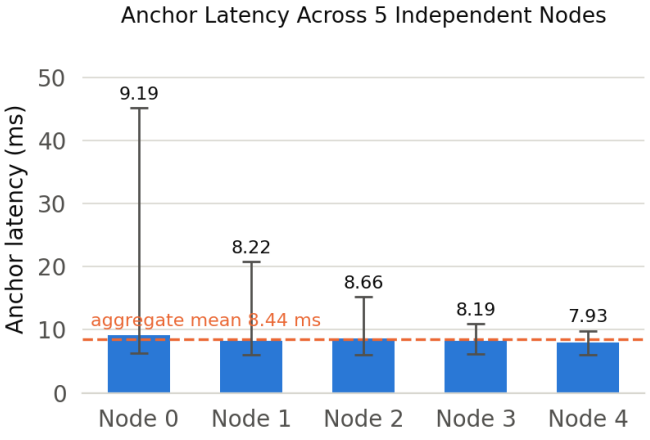


Fig. 2. Mean anchor latency per node (error bars show min-max range) across five independently-anchoring honeypot nodes, against the aggregate mean.

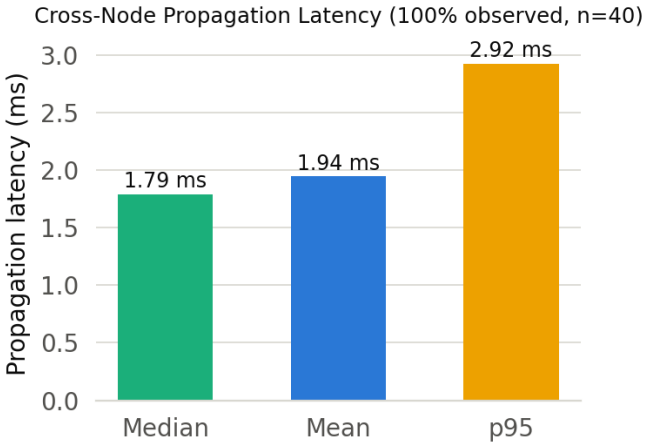


Fig. 3. Cross-node propagation latency: time from a transaction being mined to an independent, blind polling observer detecting it.

B. Cross-node propagation is fast and fully observed

In 40 trials, one process anchored a tagged IOC while a separate, blind polling process watched for it. Every trial succeeded, a 100% observed success rate. The mean interval from the transaction being mined to the observer detecting it was **1.94 ms**, with a median of 1.79 ms, a p95 of 2.92 ms, a minimum of 1.29 ms, and a maximum of 3.55 ms. The observing process holds no reference to, memory of, or channel to the writer beyond the chain, so this measures how quickly a decoupled reader picks up another party's contribution when no push mechanism exists.

C. Reconstruction and corroboration from the chain alone

We state up front what this experiment does and does not establish. It is an end-to-end correctness check on the reconstruction pipeline, not a discovery about attacker behaviour. Shared indicators are byte-identical across nodes by construction, so once decoding is correct the corroboration figures follow. What the test can still catch, and what would have made it fail, is the pipeline losing anchors, truncating or mis-parsing a tag, dropping a block, or attributing a digest to the wrong writer.

A fresh reconstruction pass, running the bootstrap a new sixth node would perform on its first day, recovered all **220** distinct IOC digests present across the five streams. The 300 anchored events collapse to 220 distinct digests once duplicates from the shared pool are accounted for, so this is **100% recall** of the full dataset using only chain history. Every digest also carried the correct writer identity, so provenance survived the round trip through raw transaction data.

Grouping by observer set separates **195** digests seen by a single node from **25** seen by two or more. Those 25 are exactly the indicators drawn from the shared campaign pool, giving **100% corroboration precision and recall** against

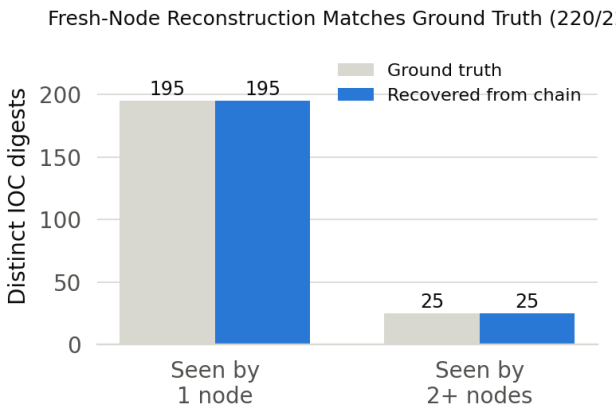


Fig. 4. Fresh-node reconstruction from chain history alone, compared against the known ground truth. The observer-set split separates single-node sightings from indicators corroborated by two or more independent nodes.

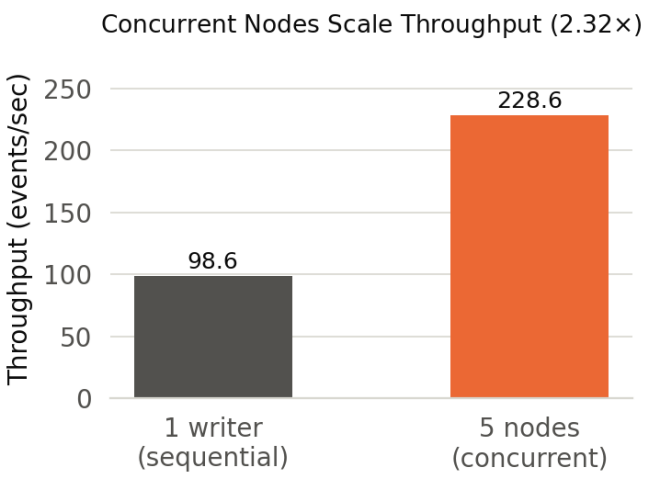


Fig. 5. Throughput measured within this harness: one sequential writer versus five federation nodes writing concurrently to the shared chain, using the same anchor-and-confirm code path on the same chain.

the known ground truth: no unrelated pair was merged, and no shared indicator was missed. The result confirms that campaign correlation, the task a centralized hub is normally built to perform, can be carried out by an uncoordinated reader scanning the chain. It does not speak to matching indicators that differ between targets, which Section 6.4 takes up.

D. A compromised node's forged local log is caught by the shared ledger

This experiment likewise verifies wiring rather than probing an adaptive adversary. The outcome follows from SHA-256 determinism and chain immutability; the test confirms the detection path is built correctly, and would have failed had provenance tagging, the local-record comparison, or the chain lookup been wrong.

We simulated an attacker breaching node0 and rewriting one of its already-anchored IOC records. The original hash (3e24abf738c9a66a...) remained on the shared chain, attributed to node0, unchanged by the simulated breach. The forged hash produced by the mutated payload (d93641e37aacf090...) appeared nowhere on-chain, because no process, honest or compromised, had anchored it. Both conditions for detection held, so the tamper was **detected via the shared ledger**.

The property this demonstrates is narrow but useful. An attacker who compromises one honeypot can rewrite that node's local evidence, yet cannot alter what the node committed before the breach, and cannot substitute a replacement that passes, because a forged record hashes to a value nobody anchored. Section 6.5 states what this leaves untested: a node that lies while writing, rather than after.

E. Concurrent writers scale rather than bottleneck

We measured both sides of this comparison in the same harness, on the same chain, through the same anchor-and-confirm code path, varying only the number of writers.

A single writer anchoring 100 events sequentially sustained **98.57 events per second**, taking 1.014 seconds. Five node workers anchoring 20 events each, again 100 events in total, confirmed all 100 in 0.437 seconds, for an aggregate **228.58 events per second**. Concurrency therefore yields a **2.32-fold** gain.

Because only the writer count differs between the two runs, the gain is attributable to concurrency rather than to any difference in code path, chain, or workload. Each worker holds its own chain account, so writers do not serialize behind a shared nonce. A shared ledger used this way does not force a network of honeypots to queue behind one writer.

F. Summary

VI. Discussion and Limitations

A. What a single local Ganache instance does and does not show

Every measurement runs against one local Ganache development chain. Ganache mines instantly and provides ordering and finality by fiat rather than through multi-validator consensus. Our propagation and throughput figures therefore contain none of the delay a real BFT or proof-of-stake round would add, and none of the variability a peer-to-peer gossip network introduces when nodes are dispersed. Moving this design to a public or consortium chain across separate organizations would reintroduce a consensus-latency term, plausibly seconds rather than milliseconds, and it would need evaluating against that chain's actual finality guarantees. We regard the contribution as the design and its local cost characterization, covering the hashing, tagging, reconstruction, and detection logic and what they add on top of whatever base chain is chosen, rather than a claim that federation is free at Internet scale.

B. Simulated nodes on one machine understate propagation latency

The five nodes are concurrent workers inside one process on one machine, talking to a single Ganache instance over localhost. The 1.94 ms mean propagation latency is thus the

TABLE II
SUMMARY OF MEASURED RESULTS AGAINST THE LIVE FIVE-NODE FEDERATION BENCHMARK.

Metric	Measured value
Simulated honeypot nodes	5
IOCs observed per node	60
Total anchored events	300
Configured overlap fraction	35%
Mean / median / p95 anchor latency	8.44 / 8.11 / 11.06 ms
Cross-node propagation trials	40
Propagation observed success rate	100%
Mean / p95 propagation latency	1.94 / 2.92 ms
Local IOCs reconstructed from chain	220 / 220 (100%)
Ground-truth shared IOCs	25
Corroborated by 2+ nodes	25
Corroboration precision / recall	100% / 100%
Compromised-node tamper detected	yes
Concurrent nodes / events attempted	5 / 100
Concurrent aggregate throughput	228.58 events/s
Single-writer baseline (this harness)	98.57 events/s

cost of a polling loop noticing a new block on a local RPC endpoint, and contains essentially no network delay. A real deployment with nodes at different organizations would see propagation dominated by the chain's block time and gossip propagation, and by each node's polling interval and network path to its RPC endpoint. Neither factor appears in a localhost benchmark. We offer the figure as a lower bound on how quickly an independent reader can notice a new anchor once it looks, not as a prediction of Internet-scale propagation.

C. Gas cost scales with the number of writing nodes

Each anchored IOC is one transaction, and on a fee-bearing chain the cost of gas is paid per anchor. Total on-chain cost therefore scales linearly with the number of participating nodes and the rate at which each observes new IOCs. A five-node network anchoring at the rate we measured pays roughly five times what one node would pay for the same per-node volume. This is an unavoidable trade against the trust benefit: centralizing at a hub is cheaper precisely because only the hub's writes, or none at all if IOCs move over a private API, incur transaction cost. A production deployment would likely batch several IOCs per node into a periodic anchor, trading immediacy for cost, as is standard practice for anchoring systems on fee-bearing chains. We anchor every IOC individually here for measurement clarity.

D. The corroboration signal is only as good as the hash input

Corroboration recognizes two nodes as having seen the same IOC when their canonical serializations are byte-identical and hash identically. Real campaign indicators are rarely identical across targets. Source addresses may differ per target within one campaign, and payloads may be varied to evade signature matching. The 100% precision and recall we report reflect a benchmark in which the shared pool is reproduced exactly across nodes. A real deployment would need a fuzzier matching layer above the exact-match primitive, perhaps hashing a normalized subset of fields or clustering near-identical payloads, and that layer's own accuracy would have to be measured separately from the chain mechanics established here.

E. No real Byzantine adversary was exercised

We simulate a compromised node's local log being rewritten after the fact and show the ledger catches the discrepancy. We do not simulate a node behaving maliciously while writing, such as one anchoring false digests to pollute the corroboration signal, or racing the chain's consensus. Defending against a dishonestly writing node is a harder problem that rests on the base chain's Sybil resistance and consensus assumptions rather than on our reconstruction logic, and it lies outside this paper's scope.

VII. Conclusion

We have presented and measured a hub-less design for cross-honeypot threat-intelligence sharing. Independently operated nodes never communicate directly, coordinate through nothing but a shared blockchain used as a passive bulletin board, and still recover deduplication, cross-node corroboration, and compromised-node forgery detection by reading that chain.

On a live five-node deployment, anchoring costs a mean of 8.44 ms per IOC with no sign of contention across nodes, and an independent reader observes a new anchor in a mean of 1.94 ms. A fresh node reconstructs the 220-IOC dataset with 100% recall and identifies all 25 shared campaign indicators with 100% corroboration precision and recall. A simulated node compromise is caught, because the ledger retains what the attacker's rewritten local log no longer shows. Five concurrent writers sustain 228.58 events per second against 98.57 for a single writer measured the same way, a 2.32-fold gain.

For a set of honeypots unwilling to place their collective intelligence with one administered hub, a shared ledger is a workable substitute. It is not a free one. The costs are gas that scales with node count and a consensus-latency term this local benchmark does not exercise, and a real deployment should weigh both rather than assume them away.

References

[1] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, "MISP: The design and implementation of a collaborative threat intelligence sharing platform," in Proc. 2016 ACM Workshop on Information Sharing and Collaborative Security (WISCS '16), Vienna, Austria, 2016, pp. 49-56.

[2] OASIS Cyber Threat Intelligence (CTI) Technical Committee, "STIX Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html

[3] OASIS Cyber Threat Intelligence (CTI) Technical Committee, "TAXII Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/taxii/v2.1/taxii-v2.1.html

[4] B. Stojkovski, G. Lenzini, V. Koenig, and S. Rivas, "What's in a Cyber Threat Intelligence sharing platform? A mixed-methods user experience investigation of MISP," in Proc. 37th Annual Computer Security Applications Conference (ACSAC '21), 2021, pp. 385-398, doi: 10.1145/3485832.3488030.

[5] C. Ruggiero and S. Bonomi, "A review of requirements and threats in blockchain-based cyber threat intelligence sharing," in Proc. IEEE 44th Int. Symp. Reliable Distributed Systems (SRDS), 2025, pp. 410-415, doi: 10.1109/SRDS69199.2025.00050.

[6] Y. Allouche, N. Tapas, F. Longo, A. Shabtai, and Y. Wolfsthal, "TRADE: TRusted Anonymous Data Exchange - threat sharing using blockchain technology," 2021, arXiv:2103.13158.

[7] H. Ali, W. J. Buchanan, J. Ahmad, M. Abubakar, M. S. Khan, and I. Wadhaj, "TrustShare: Secure and trusted blockchain framework for threat intelligence sharing," Future Internet, vol. 17, no. 7, art. 289, 2025, doi: 10.3390/fi17070289.

[8] K. Dunnett, S. Pal, and Z. Jadidi, "Challenges and opportunities of blockchain for cyber threat intelligence sharing," 2022, arXiv:2205.03822.

[9] K. Dunnett, S. Pal, G. D. Putra, Z. Jadidi, and R. Jurdak, "A trusted, verifiable and differential cyber threat intelligence sharing framework using blockchain," 2022, arXiv:2208.12031.

[10] T. Jiang, G. Shen, C. Guo, Y. Cui, and B. Xie, "BFLS: Blockchain and federated learning for sharing threat detection models as cyber threat intelligence," Computer Networks, vol. 224, art. 109604, 2023, doi: 10.1016/j.comnet.2023.109604.

[11] A. Tariq, T. Qayyum, S. Alrabaee, and M. A. Serhani, "Blockchain and distributed ledger technologies for cyberthreat intelligence sharing," 2025, arXiv:2504.02537.

[12] Y. Otoum, A. Asad, and A. Nayak, "Blockchain meets adaptive honeypots: A trust-aware approach to next-gen IoT security," 2025, arXiv:2504.16226.

[13] N. Provos, "A virtual honeypot framework," in Proc. 13th USENIX Security Symposium, San Diego, CA, USA, 2004, pp. 1-14.

[14] M. Nawrocki, M. Wahlisch, T. C. Schmidt, C. Keil, and J. Schonfelder, "A survey on honeypot software and data analysis," 2016, arXiv:1608.06249.

[15] A. Vetterl and R. Clayton, "Bitter Harvest: Systematically fingerprinting low- and medium-interaction honeypots at Internet scale," in Proc. 12th USENIX Workshop on Offensive Technologies (WOOT '18), Baltimore, MD, USA, 2018.

[16] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," ACM Transactions on Programming Languages and Systems, vol. 4, no. 3, pp. 382-401, Jul. 1982.

[17] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[18] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[19] Truffle Suite, "Ganache: A personal Ethereum blockchain for development," ConsenSys. [Online]. Available: https://archive.trufflesuite.com/ganache/